

USAGE AND COSTS

Manage costs effectively

 Copy page

Track token usage, set team spend limits, and reduce Claude Code costs with context management, model selection, extended thinking settings, and preprocessing hooks.

Claude Code charges by API token consumption. For subscription plan pricing (Pro, Max, Team, Enterprise), see claude.com/pricing. Per-developer costs vary widely based on model selection, codebase size, and usage patterns such as running multiple instances or automation.

Across enterprise deployments, the average cost is around \$13 per developer per active day and \$150-250 per developer per month, with costs remaining below \$30 per active day for 90% of users. To estimate spend for your own team, start with a small pilot group and use the tracking tools below to establish a baseline before wider rollout.

This page covers how to [track your costs](#), [manage costs for your organization](#), and [reduce token usage](#).

Track your costs

Using the `/usage` command

❗ The Session block in `/usage` shows API token usage and is intended for API users. Claude Max and Pro subscribers have usage included in their subscription, so the session cost figure isn't relevant for billing purposes. Subscribers see plan usage bars, activity stats, and a usage breakdown on the same screen.

The Session block at the top of `/usage` shows detailed token usage statistics for your current session. Claude Code computes the dollar figure locally from token counts

```
Total cost:                $0.55
Total duration (API):      6m 20s
Total duration (wall):     6h 33m 10s
Total code changes:        0 lines added, 0 lines removed
Usage by model:
  claude-sonnet-4-6: 1.2k input, 5.3k output, 940.0k cache read, 50.0k cache write
```

These totals reset when `/clear` starts a new session, so the next session's total cost starts at \$0. Before v2.1.211, they kept accumulating across `/clear` for the lifetime of the Claude Code process.

Plan usage breakdown

On a Pro, Max, Team, or Enterprise plan, `/usage` also shows a breakdown of what counts against your plan limits:

- **Attribution:** recent usage attributed to skills, subagents, plugins, and individual MCP servers, each shown as a percentage of the total. An MCP server's share counts only the requests that consumed one of its tool results. Before v2.1.222, after one call to an MCP server, Claude Code attributed every subsequent request to that server, overstating its share.
- **Behavior flags:** behaviors such as long context or cache misses, flagged when one accounts for 10% or more of recent usage.
- **Loops:** a row for each of the heaviest `/loop` **or other scheduled tasks** that ran recently, ordered by total tokens, with a count of the rest. Claude Code reports how often each task fires, how many times it ran, its total and per-run tokens, and when it last ran. Claude Code keys a row by the task's prompt, so a loop you stop and re-create stays one row. Requires Claude Code v2.1.242 or later.

Press `d` or `w` to switch between the last 24 hours and the last 7 days. The figures are approximate and computed from local session history on this machine, so usage from other devices or `claude.ai` is not included.

Check your usage-credits spend

`/usage` also shows a usage-credits row while usage credits are on. What the row shows depends on your plan:

- **Pro and Max:** your spend for the current month, measured against your monthly spend limit when you have set one. When you haven't set a limit, the row shows `Unlimited` and no spend figure.
- **Team and Enterprise:** your own spend for the current month, measured against any limit your organization set that applies to you. A limit that covers the whole organization doesn't appear in the row. When you have no limit of your own, the row shows your spend with no limit beside it. While usage credits are off for you, `/usage` shows no usage-credits row.

When you have a spend limit, the row appears as soon as usage credits are on and shows 0% until you first spend usage credits. Before v2.1.236, `/usage` showed the row only on Pro and Max plans, and a row with a spend limit stayed hidden until you had spent something.

When the usage request fails

When the request for your plan limits fails, most often because the usage endpoint is rate limited, `/usage` shows the last usage bars it loaded on this machine within the past 60 minutes, along with a `Showing last-known usage` note stating how long ago that data was fetched. Press `r` to retry; a successful retry replaces the last-known bars with fresh data. Without a snapshot from the past 60 minutes, `/usage` reports that the usage endpoint is rate limited and offers the same retry shortcut. Before v2.1.208, a rate-limited request in a session that hadn't loaded usage yet always showed the error with no bars.

suggestions for using Claude Code more effectively. A single run analyzes up to 200 sessions it hasn't seen before and skips very short ones. When sessions are left out, the report header shows the analyzed count with the total in parentheses, for example `200 sessions (412 total)`.

Claude Code writes the latest report to `~/.claude/usage-data/report.html` and saves a timestamped copy of each run in the same directory, so earlier reports aren't overwritten. Claude Code deletes reports on the same schedule as the rest of your session data: at startup, it removes files older than `cleanupPeriodDays`, 30 days by default.

You can run `/insights` on any plan and with any provider. The analysis runs through the same provider and account as your regular sessions, and the tokens count against your plan or API usage. Sessions from other devices and `claude.ai` aren't included.

Add usage credits to your subscription

Usage credits let you keep working past your plan's usage limit. To manage them, run `/usage-credits` after signing in with your `claude.ai` subscription through `/login`; the command isn't available with API key authentication. What it opens depends on your role:

Your role	What <code>/usage-credits</code> does
Pro or Max subscriber	Opens <u>Settings > Usage</u> on <code>claude.ai</code> in the browser. In its Usage credits section you can turn usage credits on or off and check your credit balance, this month's spend, and your monthly spend limit
Team or Enterprise member with billing access	Opens your organization's usage settings, <u>Admin settings > Usage</u> , in the browser
Team or Enterprise member without billing access	Asks you to confirm, then sends a request to your organization's admins. Before v2.1.211, Claude Code sent the request without a confirmation step

If you run `/usage-credits` again while your earlier request is waiting on an admin, Claude Code tells you a request has already been sent rather than sending a duplicate. After an admin dismisses your request, running the command again sends a new one. Before v2.1.222, a dismissed request also blocked new requests.

On Pro and Max plans, when you reach your spend limit with usage credits still available, Claude Code prompts you to raise or remove the limit without leaving the CLI. If the server rejects the change, see [Could not update your spend limit](#).

Manage costs for your organization

Which controls you have depends on how your organization accesses Claude Code: a Claude for Teams or Enterprise plan, the Claude Console, or a cloud provider. On Teams and Enterprise plans, usage draws from each member's seat allowance. On the Console and on cloud providers, usage is billed per token to your organization. If your organization mixes sign-in methods, each developer is metered according to the one they authenticated with.

The table maps each setup to where you see spend, where you cap it, and how you pull per-user numbers. On an individual Pro or Max plan you have no organization to manage, so track your own usage-credit spend, including [fast mode](#), under [Add usage credits to your subscription](#).

Your setup	See spend	Cap spend	Per-user reporting
Claude for Teams or Enterprise	Spend report in org analytics	Spend limits in admin settings	Spend report CSV ; Enterprise Analytics API on Enterprise
Claude Console (API)	Console usage page	Workspace spend limits	Console dashboard , Claude Code Analytics API

OpenTelemetry export works on every setup and is the only option that streams per-user token and cost metrics into your own observability stack in near real time.

Claude for Teams and Enterprise

On Claude for Teams and Enterprise plans, each member's Claude Code usage draws from a per-seat allowance that resets on a rolling five-hour window and a weekly window. The allowance is shared with Claude chat and Cowork, and its size depends on the member's **seat tier** (Standard or Premium). Your controls live in the [claude.ai](#) admin console, not the Claude Console.

- **See spend:** the **spend report in org analytics** shows estimated spend per user and per model, with CSV export, updated daily. The report covers usage-credit spend and appears once usage credits are turned on. Usage inside the seat allowance isn't metered in dollars.
- **See adoption:** the **analytics dashboard** shows daily active users, sessions, and contribution metrics, with CSV export of contribution data. See **track team usage with analytics**.
- **Cap spend:** the seat allowance is the default ceiling. To let members continue past it, turn on **usage credits** and set spend limits at the organization, group, or individual member level.
- **Pull per-user numbers:** on the Enterprise plan, the **Enterprise Analytics API** returns per-user usage and cost reports across Claude surfaces, including Claude Code. A Primary Owner creates a key with the `read:analytics` scope at **claude.ai/analytics/api-keys**. On the Teams plan, export the **spend report CSV**, which lists token usage and estimated spend per user and per model.

The **Claude Enterprise consumption guide** is the planning reference for admins. It explains how consumption differs across Claude chat, Claude Code, and Cowork, and gives per-user dollar starting points for budgeting. Budget more for a coding seat than a

API organizations manage Claude Code spend through [workspaces](#). You can [set workspace spend limits](#) on total Claude Code spend and [view cost and usage reporting](#) in the Console.

ⓘ When you first authenticate Claude Code with your Claude Console account, a workspace called “Claude Code” is automatically created for you. This workspace provides centralized cost tracking and management for all Claude Code usage in your organization. You cannot create API keys for this workspace; it is exclusively for Claude Code authentication and usage.

For organizations with custom rate limits, Claude Code traffic in this workspace counts toward your organization's overall API rate limits. You can set a [workspace rate limit](#) on this workspace's Limits page in the Claude Console to cap Claude Code's share and protect other production workloads.

For per-user reporting, the [Console dashboard](#) shows spend and accepted lines per member, and the [Claude Code Analytics API](#) returns the same daily per-user metrics programmatically with an [Admin API key](#). See [analytics for API customers](#).

Rate limit recommendations

When setting up Claude Code for teams, consider these Token Per Minute (TPM) and Request Per Minute (RPM) per-user recommendations based on your organization size:

Team size	TPM per user	RPM per user
1-5 users	200k-300k	5-7
5-20 users	100k-150k	2.5-3.5
20-50 users	50k-75k	1.25-1.75
50-100 users	25k-35k	0.62-0.87
100-500 users	15k-20k	0.37-0.47
500+ users	10k-15k	0.25-0.35

code concurrently in larger organizations. These rate limits apply at the organization level, not per individual user, which means individual users can temporarily consume more than their calculated share when others aren't actively using the service.

❗ If you anticipate scenarios with unusually high concurrent usage (such as live training sessions with large groups), you may need higher TPM allocations per user.

Cloud providers

On Amazon Bedrock, Google Cloud's Agent Platform, and Microsoft Foundry, Claude Code is billed per token to your cloud account, and spend controls live in your cloud provider's billing console. Claude Code does not send metrics from your cloud back to Anthropic, so the [analytics dashboards](#) and the Claude Code Analytics API do not cover this usage.

For per-user cost attribution, you have three options:

- **OpenTelemetry:** [export metrics](#) from each developer's machine to your own observability stack. This gives you per-user token counts, costs, and tool activity regardless of provider.
- **A Claude apps gateway:** a self-hosted [Claude apps gateway](#) provides per-user usage attribution, OTLP metrics with token counts, and [per-user spend limits](#) on these providers.
- **An LLM gateway:** route all Claude Code traffic through a proxy that tracks spend per key. Several large enterprises reported using [LiteLLM](#), an open-source tool that [tracks spend by key](#). This project is unaffiliated with Anthropic and has not been audited for security.

When a developer asks about a limit

Developers usually bring limit questions to their admin, so it helps to know which ceiling they hit. The four situations mean different things:

your Sonnet limit” message, switching to a model outside that family with `/model` does keep the developer working. See [usage limit errors](#). What the developer can do in the meantime:

- Run `/usage-credits` to request usage beyond the allowance, if you have [usage credits](#) turned on.
- On Claude Code v2.1.234 or later, [wait and continue the interrupted task automatically after the reset](#); that section lists when Claude Code starts the wait on its own and when the developer picks it from `/rate-limit-options`. To control for your fleet whether Claude Code starts that wait on its own, set `autoContinueAtUsageLimit` in [managed settings](#).
- **A spend limit message from a Claude apps gateway:** the developer passed a spend cap you set on your self-hosted gateway, and the gateway blocks their requests until the period resets or you raise the cap. See [gateway spend limits](#) for caps, reset schedules, and the message the developer sees.
- **A context or auto-compact warning:** not a usage limit. The conversation has grown close to the session's [auto-compact window](#), the threshold where Claude Code summarizes older history to free space. Point the developer at [reduce token usage](#).
- **Unexpectedly high spend on an API or cloud-provider plan:** usually traces back to long sessions that were never cleared or to Opus left as the default model. The highest-impact habits to share are clearing between unrelated tasks and matching the model to the job, both covered in [reduce token usage](#).

Agent team token costs

[Agent teams](#) spawn multiple Claude Code instances, each with its own context window. Token usage scales with the number of active teammates and how long each one runs.

To keep agent team costs manageable:

- Use Sonnet for teammates. It balances capability and cost for coordination tasks.

start.

- Shut down teammates when their work is done. Each active teammate continues consuming tokens until it exits or the session ends.
- Agent teams are disabled by default. Set `CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1` in your [settings.json](#) or environment to enable them. See [enable agent teams](#).

Reduce token usage

Token costs scale with context size: the more context Claude processes, the more tokens you use. Claude Code automatically optimizes costs through [prompt caching](#), which reduces costs for repeated content like system prompts, and auto-compaction, which summarizes conversation history when approaching context limits.

The following strategies help you keep context small and reduce per-message costs.

Manage context proactively

Use `/usage` to check your current token usage, or [configure your status line](#) to display it continuously.

- **Clear between tasks:** Use `/clear` to start fresh when switching to unrelated work. Stale context wastes tokens on every subsequent message. Use `/rename` before clearing so you can easily find the session later, then `/resume` to return to it.
- **Add custom compaction instructions:** `/compact Focus on code samples and API usage` tells Claude what to preserve during summarization. In a fresh session, `/compact` prints `Not enough messages to compact.` because there's no conversation history to summarize yet.

You can also customize compaction behavior in your `CLAUDE.md` file at the root of your project:

Choose the right model

Sonnet handles most coding tasks well and costs less than Opus. Reserve Opus for complex architectural decisions or multi-step reasoning. Use `/model` to switch models mid-session, or set a default in `/config`. For simple subagent tasks, specify `model: haiku` in your subagent configuration.

Reduce MCP server overhead

MCP tool definitions are deferred by default, so only tool names and server instructions enter context until Claude uses a specific tool. Run `/context` to see what's consuming space.

- **Prefer CLI tools when available:** Tools like `gh`, `aws`, `gcloud`, and `sentry-cli` are still more context-efficient than MCP servers because they don't add any per-tool listing. Claude can run CLI commands directly.
- **Disable unused servers:** Run `/mcp` to see configured servers and disable any you're not actively using.

Install code intelligence plugins for typed languages

Code intelligence plugins give Claude precise symbol navigation instead of text-based search, reducing unnecessary file reads when exploring unfamiliar code. A single "go to definition" call replaces what might otherwise be a `grep` followed by reading multiple candidate files. Installed language servers also report type errors automatically after edits, so Claude catches mistakes without running a compiler.

Offload processing to hooks and skills

Custom hooks can preprocess data before Claude sees it. Instead of Claude reading a 10,000-line log file to find errors, a hook can `grep` for `ERROR` and return only matching lines, reducing context from tens of thousands of tokens to hundreds.

For example, this PreToolUse hook filters test output to show only failures:

settings.json **filter-test-output.sh**

Add this to your **settings.json** to run the hook before every Bash command:

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "~/.claude/hooks/filter-test-output.sh"
          }
        ]
      }
    ]
  }
}
```

To verify the setup, run `/hooks` and check that the hook appears under PreToolUse. You can also start Claude Code with `claude --debug` and run a test command such as `npm test`. The debug log shows `modified tool input keys: [command]` when the hook rewrites the command.

Move instructions from CLAUDE.md to skills

Your **CLAUDE.md** file is loaded into context at session start. If it contains detailed instructions for specific workflows (like PR reviews or database migrations), those tokens are present even when you're doing unrelated work. **Skills** load on-demand only

Extended thinking is enabled by default because it significantly improves performance on complex planning and reasoning tasks. Thinking tokens are billed as output tokens, and the default budget can be tens of thousands of tokens per request depending on the model. For simpler tasks where deep reasoning isn't needed, you can reduce costs by lowering the **effort level** with `/effort` or in `/model`, disabling thinking in `/config`, or, on models with a **fixed thinking budget**, lowering the budget by setting the `MAX_THINKING_TOKENS` **environment variable**, for example `MAX_THINKING_TOKENS=8000`. Adaptive-reasoning models ignore nonzero budgets, so use effort levels there instead. Disabling thinking is not available on Fable 5, which always uses extended thinking.

Delegate verbose operations to subagents

Running tests, fetching documentation, or processing log files can consume significant context. Delegate these to **subagents** so the verbose output stays in the subagent's context while only a summary returns to your main conversation.

Manage agent team costs

Agent teams use approximately 7x more tokens than standard sessions when teammates run in plan mode, because each teammate maintains its own context window and runs as a separate Claude instance. Keep team tasks small and self-contained to limit per-teammate token usage. See **agent teams** for details.

Write specific prompts

Vague requests like "improve this codebase" trigger broad scanning. Specific requests like "add input validation to the login function in auth.ts" let Claude work efficiently with minimal file reads.

- **Use plan mode for complex tasks:** Press Shift+Tab to cycle to plan mode before implementation. Claude explores the codebase and proposes an approach for your approval, preventing expensive re-work when the initial direction is wrong.
- **Course-correct early:** If Claude starts heading the wrong direction, press Escape to stop immediately. Use `/rewind` or double-tap Escape to restore conversation and code to a previous checkpoint.
- **Give verification targets:** Include test cases, paste screenshots, or define expected output in your prompt. When Claude can verify its own work, it catches issues before you need to request fixes.
- **Test incrementally:** Write one file, test it, then continue. This catches issues early when they're cheap to fix.

Background token usage

Claude Code uses tokens for some background functionality even when idle:

- **Conversation summarization:** Background jobs that summarize previous conversations for the `claude --resume` feature
- **Command processing:** Some commands like `/usage` may generate requests to check status

These background processes consume a small amount of tokens (typically under \$0.04 per session) even without active interaction.

Why usage climbs in a long session

A session that has been open for hours can use far more of your plan limits than your activity suggests, usually for one of these reasons:

usage for the whole conversation. See [Manage context proactively](#) for ways to keep your context small

- **Cache misses:** your first message after a break longer than the [cache lifetime](#) misses the cache and reprocesses your full context. The lifetime is an hour on a subscription and drops to five minutes once you're drawing on [usage credits](#); on an API key or cloud provider, it's five minutes by default. To keep the one-hour lifetime while drawing on usage credits, [choose the TTL yourself](#). On Pro and Max plans, when you resume a large session after a long break, Claude Code [offers to resume from a summary](#) so later requests don't carry the full history
- **Scheduled tasks:** a [scheduled task](#) fires on its interval even while the session is idle, sending your full context each time
- **Cross-session messages:** Claude Code delivers a [message from another of your sessions](#) as a new turn when this session sits idle, sending your full context each time. To hold inbound messages instead of delivering them, set `crossSessionInbound` to `hold`
- **Goal check-ins:** while background work keeps an active [goal](#) waiting, Claude Code [asks Claude to check on that work](#) even when the session sits idle, starting a new turn that sends your full context. Claude Code starts at most three idle check-ins per goal between your prompts. Before v2.1.246, idle check-ins were uncapped. To turn check-ins off, set `CLAUDE_CODE_GOAL_CHECKIN_MINUTES` to `0`. Idle check-ins require Claude Code v2.1.236 or later
- **Agent teammates:** each active [teammate](#) keeps consuming tokens until it exits
- **Compaction:** `/compact` reads the conversation it summarizes, so [compacting a large context](#) is itself a large request. When you want a fresh start instead of continuity, `/clear` costs nothing

On a Pro, Max, Team, or Enterprise plan, the `/usage` breakdown flags behaviors that account for 10% or more of your recent usage, such as long context or cache misses, each with a tip to reduce it.

For billing questions about your specific account, contact Anthropic support through the in-product messenger:

- **Subscription plans** (Pro, Max, Team, Enterprise): sign in at claude.ai, click your initials in the lower left, and select **Get help**
- **Console (API) billing**: sign in at platform.claude.com, click your initials, and select **Get help**

See [How to get support](#) for the full flow, including who can reach a human agent on each plan.

Was this page helpful?

 Yes

 No

Company

[Anthropic](#)

[Careers](#)

[Economic Futures](#)

[Research](#)

[News](#)

[Trust center](#)

[Transparency](#)

Help and security

[Availability](#)

[Status](#)

[Support center](#)

[Customer stories](#)

[Engineering blog](#)

[Events](#)

[Powered by Claude](#)

[Service partners](#)

[Startups program](#)

[Disclosure policy](#)

[Usage policy](#)

[Commercial terms](#)

[Consumer terms](#)